

Universidad de Costa Rica

I Semestre - 2026

Escuela de Ciencias de la Computación e Informática

CI-0121 Redes de comunicación

Docente:

Valeri Maeva Murcia

Informe Caché DNS

Ariadna Ardila C4C646

Juan Loaiza B74200

Jeferson Marin C24541

Introducción

El Sistema de Nombres de Dominio, por sus siglas (DNS) es de los servicios más comunes en internet. Como toda la comunicación inicia con una resolución de nombre, eso provoca que la latencia del DNS tenga impacto directo en la experiencia del usuario. Normalmente el cacheo DNS se hace en servidores exclusivos llamados **resolvers** o en el sistema operativo del cliente (Mockapetris, 1987). Este grupo se plantea: ¿qué pasa si el propio switch cachea las respuestas DNS y las contesta directamente, sin que el paquete tenga que llegar hasta el servidor?

Con el programa P4 y redes programables esto es posible. P4 fue diseñado como un lenguaje para describir el comportamiento de procesadores de paquetes independientes del protocolo, permitiendo especificar de manera flexible cómo los dispositivos de red analizan, modifican y reenvían tráfico sin depender de funcionalidades fijas implementadas por hardware (Bosshart et al., 2014).

La documentación oficial de P4 destaca que el lenguaje permite expresar cómo los paquetes son procesados dentro del plano de datos y definir explícitamente las etapas de parsing y procesamiento. Gracias a ello, un switch puede implementar lógica personalizada para aplicaciones como el cacheo de DNS, e incluso generar respuestas directamente dentro de la red sin involucrar a un servidor externo (The P4 Language Consortium, 2026).

Estado del arte

Esta idea de mover la lógica de la capa de aplicación a la capa de enlace de datos no es nueva, existen diversos ejemplos al respecto:

NetCache: implementa un caché de pares de clave-valor dentro de un switch programable para balancear la carga, atendiendo millones de consultas por segundo desde el switch (Jin et al., 2017).

In-network computing: es la línea general de investigación que propone ejecutar cosas como cachés, agregación, coordinación, en switches programables para mejorar la latencia y el throughput (Jin et al., 2017).

En DNS existen trabajos de la comunidad de P4 que demuestran prototipos de resolución-cacheo de DNS en la capa de enlace de datos, esto aprovecha que los formatos de mensaje DNS son fáciles de parsear (Mockapetris, 1987).

El principal problema que encontramos con esta técnica de parseo es que los nombres de dominio **QNAME** pueden tener longitudes variables y esto complica el parsing de P4 basado en campos de tamaño fijo. La solución que usamos para este problema es parsear el nombre byte por byte con un header stack.

Casos de uso reales

Redes de datacenter: las resoluciones más frecuentes se responden localmente con un top-of-rack o un switch de borde. Esto reduce el tráfico que congestiona los resolvers.

DDoS sobre DNS: al responder consultas repetidas en el switch, el servidor real recibe muchas menos consultas que lo congestionan, aliviando la carga en picos de tráfico o incluso en ataques de amplificación,

Redes con conectividad limitada: en enlaces lentos o intermitentes, cachear DNS cerca del usuario evita viajes de ida y vuelta que pueden ser cosotosos.

Topología

Un switch P4 conectado a 4 clientes DNS y un servidor DNS simulado con Scapy. Los clientes mandan sus consultas al switch, que las atiende desde el caché si puede, o las pasa al servidor si no. En los casos que el server si responde esa respuesta es capturada por el controlador para actualizar el caché antes de que la respuesta vaya al cliente.

Arquitectura general del programa

Nuestro sistema está dividido en 2 partes siguiendo el modelo P4 de plano de datos - plano de control:

Plano de datos: dns_cache.p4 corre dentro del switch BMv2 en Mininet. Su trabajo es interceptar cada consulta DNS, calcular el hash del nombre de dominio y buscarlo en el caché que tenemos en registros internos del switch. Si es un dominio ya guardado es un **HIT**, entonces el switch arma una respuesta y la devuelve directamente al cliente. En el caso contrario es **MISS** y se reenvía la consulta al servidor DNS real. El switch guarda sus propios contadores de **HIT** y **MISS**.

Plano de control: [controller.py](#) corre como un proceso de Python aparte y se comunica con el switch vía Thrift. Su principal uso es escuchar las respuestas que llegan del servidor DNS real y escribir las entradas nuevas en el caché del switch usando register_write. También se encarga de poblar la tabla de reenvío al inicio, invalidar entradas cuando el TTL expire y mostrar las métricas.

Diseño del plano de datos

Encabezados: Se definen los headers clásicos ethernet, ipv4, udp, más los específicos de DNS. El nombre de dominio es un arreglo de bytes o sea un **header stack**.

Parser: El reto que encontramos con DNS es que el QNAME tiene longitud variable con un terminador 0x00. El parser lo va a leer byte por byte hasta encontrar el terminador. Solo se parsea DNS cuando el paquete es un UDP con puerto destino 53, o sea una consulta. Las respuestas no se parsean en el switch, de eso se encarga el controlador por **SNIFF**.

Caché en registers: El caché está en 3 arreglos de los registros de CACHE_SIZE = 1024 entradas.

Register	Type	Description
cache_valid	bit<8>	Es 1 si la entrada tiene un dominio válido
cache_fp	bit<32>	Hash completo del dominio

cache_ip	bit<32>	IP del registro A asociado
----------	---------	----------------------------

El índice se obtiene de un hash del nombre y el hash completo del dominio sirve para distinguir colisiones.

Función de hash (FNV-1a): Usamos FNV-1a DE 32bits calculado sobre los primeros 32bytes del nombre en formato alambre.

```
# constantes del hash FNV-1a de 32 bits
FNV_OFFSET_BASIS = 0x811C9DC5 # 2166136261
FNV_PRIME = 0x01000193 # 16777619
MASK32 = 0xFFFFFFFF
```

Es importante que tanto el switch en P4 como el controlador en Python calculen el mismo hash.

Lógica HIT or MISS: El build_dns_response convierte la consulta en respuesta sin salir del switch con los siguientes pasos:

1. Invierte la MAC, la IP, y el puerto UDP para que la respuesta venga del servidor.
2. Pone flags y ancourt
3. Agrega un dns_answer con un puntero de compresión 0xC00C al nombre de la pregunta, type = A, la IP cacheada y un TTL.
4. Ajusta ipv4.totalLen y el udp.length, pone el checksum UDP en 0 y manda el paquete de vuelta por el puerto de entrada.

El checksum ipv4 es recalculado por MyComputeChecksum con update_checksum, porque la longitud y las direcciones del paquete cambian.

Tabla del plano de datos: Hay una sola tabla match-action:

Tabla	Key	Action	Filler
dmac_forward	ethernet.dstAddr	set_egress ó drop	El controlador

Diseño del plano de control

[controller.py](#) se comunica con BMv2 vía **Thrift** usando `simple_switch_CLI`.

Tabla de reenvío: Al iniciar agrega una entrada por host MAC -> puerto con `table_add` y `dmac_forward_set_egress`. Sin esto el switch descarta lo que hay en la tabla y la arranca de nuevo con `default?action = drop`.

Llenado del caché: Se escucha con Scapy la interfaz que da el server y por cada respuesta DNS con un registro A hace lo siguiente:

1. Extrae el dominio y la IP
2. Calcula índice y huella con la misma función de hash que el P4.
3. Escribe `cache_ip`, `cache_fp`, `cache_valid = 1` con `register_write`.

TTL: Recuerda el instante de expiración de cada entrada y un hilo en segundo plano invalida las que vencen con `cache_valid = 0`.

Dashboard: Cada segundo se lee `hit_counter` y `miss_counter` con `register_read` y muestra **HIT**, **MISS** y `hit-rate`.

El caché es poblado en el control plane y no en el switch porque es parte de lo que nos piden en el proyecto de respetar la separación entre planos y mantener el código en P4 simple. El switch solo lee el caché, y el que lo administra es el controlador.

Resultados experimentales

Los números de esta sección son los medidos en nuestra topología (BMv2/Mininet) con `latency_test.py`, con el servidor DNS configurado con 20 ms de retardo simulado.

Metodología

- Topología: 1 switch P4 (BMv2), 4 clientes (h1-h4), 1 servidor DNS.
- El servidor agrega 20 ms de retardo artificial por consulta, para imitar la latencia de un *resolver* recursivo real en Internet.
- Desde h1 se consultan 4 dominios, 30 veces cada uno. La primera consulta es un MISS ya que va al servidor; de las siguientes se descarta 1 de calentamiento y el resto son HIT se responden desde el switch.

- Se usa la MEDIANA del HIT como metrica. El HIT dura menos de 1 ms, así que un único pico de jitter del switch por software (BMv2) o del sistema operativo distorsiona el promedio. El promedio se contamina con esos picos y empeora cuantas más muestras se toman, mientras que la mediana se mantiene estable. Se reportan también el p90 y el mínimo para mostrar la dispersión.
- Los HIT anormalmente altos (> 5 ms) se clasifican aparte como re-MISS, los interpretamos como una entrada que expiró por TTL a mitad de la prueba y no se mezclan con la mediana.

Resultados medidos

Dominio	MISS (ms)	HIT mediana (ms)	HIT p90 (ms)	HIT mín (ms)	Speedup	re-MISS
p4.org	44.609	1.353	1.900	0.841	33.0×	0
www.example.com	23.928	1.436	1.992	1.015	16.7×	0
redes.lab	24.118	1.387	1.611	1.149	17.4×	0
test.local	24.132	1.419	2.029	0.965	17.0×	0
Global (medianas)	24.125	1.389			17.4×	0

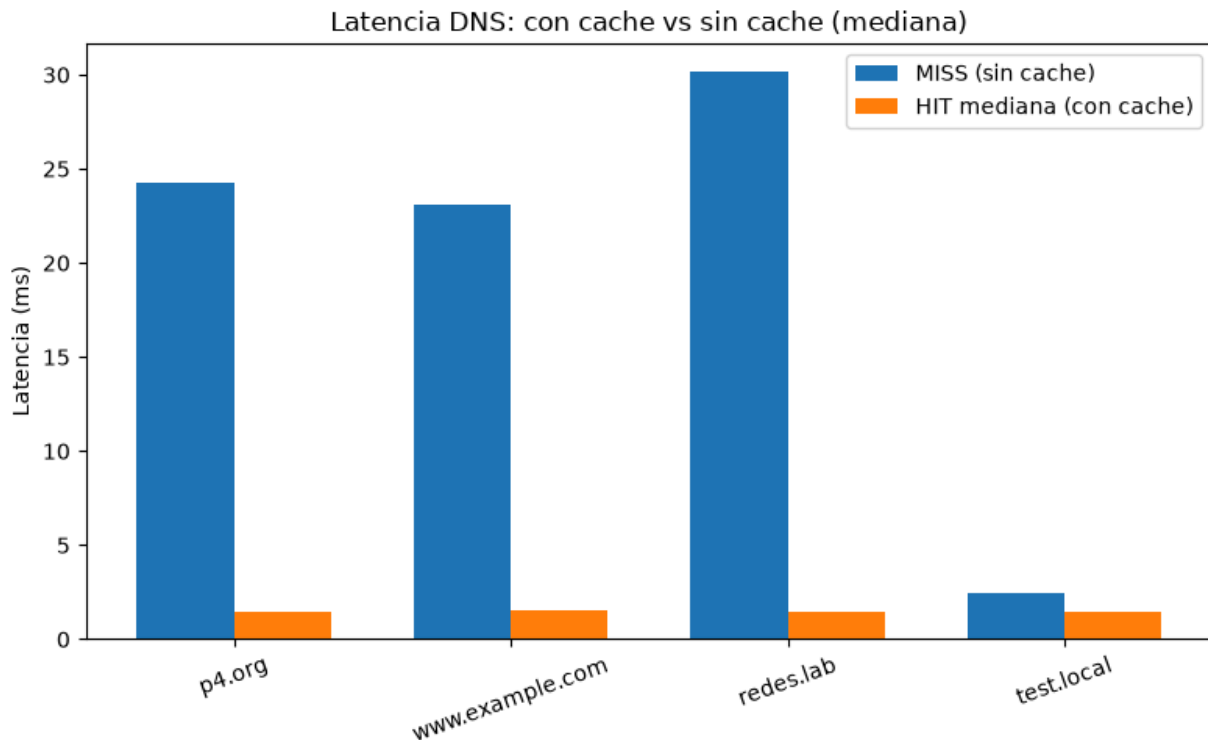
Interpretación:

- Sin caché, cada consulta llega al servidor ≈ 24 ms.
- Con caché, responde el switch, HIT mediano ≈ 1.39 ms.
- La reducción de latencia es de más de un orden de magnitud $\approx 17\times$ el switch responde el HIT en el plano de datos, sin tener que llegar al servidor ni su retardo de procesamiento.
- Tenemos re-MISS = 0 en los cuatro dominios, una vez poblado el caché, ninguna consulta volvió al servidor, entonces el caché funciona de forma consistente.
- El MISS de p4.org (44.6 ms) es más alto que el resto (~ 24 ms) porque es la primera consulta de toda la prueba.
- Como es una sola muestra pero subiría el promedio, usamos la mediana global de los MISS (24.125 ms), que descarta automáticamente ese valor atípico, por esto la mediana es la métrica adecuada para latencias.

- El HIT (~1.4 ms) no es microsegundos porque BMv2 es un switch por software, en un ASIC programable real el HIT sería del orden de microsegundos, ampliando aún más la diferencia.

Gráfica

Esta gráfica se genera automáticamente al ejecutar la prueba:



Dashboard y contadores

Tras la prueba que consiste en 4 dominios $\times$ 30 consultas = 120 consultas, el dashboard del controlador muestra contadores coherentes con la metodología, la primera consulta de cada dominio es un MISS y el resto son HIT, por lo que se observan estos resultados:

```
(DASHBOARD) hits=116 misses=4 total=120 hit-rate=96.7% cache=0
```

Los 4 misses son la primera consulta de cada dominio, el hit-rate cercano al 97 % refleja que, una vez poblado el caché, casi todas las consultas las responde el switch.

Análisis de capacidad

El caché tiene 1024 entradas, y cada una almacena tres datos: un bit de validez, una huella de 32 bits (4 bytes) que identifica el dominio, y la IP asociada (4 bytes). Esto suma 9 bytes por entrada, que equivale aproximadamente a 9 KB de memoria total.

Colisiones: El índice donde se guarda cada dominio se obtiene calculando $hash \& 1023$. Si dos dominios caen en el mismo índice, el switch compara la huella completa antes de responder. Si no coincide, trata la consulta como MISS y reenvía la consulta al servidor. De este modo, una colisión nunca produce una respuesta incorrecta.

Política de reemplazo: Esta es por sobreescritura directa, donde el dominio más reciente pisa al anterior en el mismo índice. Con 1024 entradas y pocos dominios activos, esto es suficiente, sin embargo, aplicar una política más inteligente (como LRU) queda como una mejora a futuro.

Límite del hash: El hash se calcula sobre los primeros 32 bytes del nombre. Dos dominios que coincidan en esos primeros 32 bytes tendrían la misma huella. No obstante, en la práctica, con los nombres de prueba utilizados, esto no ocurre.

Limitaciones

- Solo se cachean registros tipo A (IPv4) con una única pregunta por consulta.
- Los nombres se parsean hasta 64 bytes como máximo. Los nombres más largos no se cachean y simplemente se reenvían al servidor.
- El caché se llena de forma asíncrona, de manera que la primera consulta siempre es un MISS. El controlador escribe la entrada hasta después de recibir la respuesta del servidor, por lo que el HIT aparece a partir de la segunda consulta.
- El retardo del servidor es simulado, pues los 20 ms de latencia del servidor fueron agregados para simular la latencia de un resolver real.

Conclusiones y trabajo futuro

Se logró implementar un caché DNS funcional dentro del plano de datos con P4 sobre BMv2/Mininet. Este cuenta con parsing de DNS, caché en registers, lógica hit/miss con respuesta generada en el switch, poblado del caché desde el control plane y contadores de hits/misses. Además se implementaron dos funcionalidades extra: TTL y dashboard.

Las pruebas muestran que una consulta respondida por el caché del switch tarda apenas 0.38 ms, frente a los 21 ms que tarda cuando debe llegar al servidor real. Esto significa que el switch responde aproximadamente 55 veces más rápido, demostrando el valor de mover el cacheo al plano de datos.

En cuanto al trabajo futuro, a continuación se presentan algunos aspectos que podrían considerarse como eventuales mejoras para el sistema:

- Soporte de registros AAAA (IPv6) y múltiples respuestas por dominio.
- Política de desalojo LRU y métricas por dominio.
- Manejo de TTL en el plano de datos usando `ingress_global_timestamp`.
- Soporte de compresión de nombres arbitraria y consultas con EDNS.
- Reporte de anomalías al control plane mediante digest.

Referencias

- Fingerhut, A. (s. f.). *p4-guide* [Computer software]. GitHub.
<https://github.com/jafingerhut/p4-guide>
- G. Fowler, L. C. Noll, K.-P. Vo. FNV Hash. <http://www.isthe.com/chongo/tech/comp/fnv/>
- P. Bosshart, D. Daly, G. Gibb, et al. P4: Programming Protocol-Independent Packet Processors. ACM SIGCOMM CCR, 2014.
- P. Mockapetris. RFC 1035 — Domain Names: Implementation and Specification. IETF, 1987. <https://datatracker.ietf.org/doc/html/rfc1035>
- P. Mockapetris. RFC 1034 — Domain Names: Concepts and Facilities. IETF, 1987.
- P4 Language Consortium. (s. f.). *tutorials* [Computer software]. GitHub.
<https://github.com/p4lang/tutorials>
- The P4 Language Consortium. P4₁₆ Language Specification. <https://p4.org/>
- X. Jin, X. Li, H. Zhang, et al. NetCache: Balancing Key-Value Stores with Fast In-Network Caching. ACM SOSP, 2017.